17 攤銷分析

Part IV Advanced Design and Analysis Techniques

本章定位

資料結構的一次操作可能很昂貴，整段序列卻仍有線性總成本。本章以stack、counter與動態表推導三種攤銷方法，核心是初始信用、每個前綴的帳目，以及擴縮交界是否仍能支付搬移。

前置知識

序列操作與總成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 區分攤銷成本與平均成本。
- 用 aggregate、accounting、potential 三法分析同一結構。
- 證明動態擴張表在加倍策略下 O(1) 攤銷。
- 17.1 Aggregate analysis
- 17.2 The accounting method
- 17.3 The potential method
- 17.4 Dynamic tables

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.1 Aggregate analysis

攤銷分析對任意合法操作序列給總成本上界。若從固定初態出發，任意長度 m 的序列都滿足 Σc_i≤Am+B，就能說每次操作有 O(1) 攤銷成本。這不表示每個 c_i 都是常數，也沒有對隨機輸入取平均。

以空 stack 的 PUSH、POP、MULTIPOP(k) 為例。PUSH 成本1，成功 POP 成本1，MULTIPOP 移除 q=min(k,size) 項，成本 q；呼叫與空測試的固定開銷另加 O(1)。

每個元素只被推入一次、彈出至多一次，所以整段序列的彈出工作不超過 PUSH 次數。m 次操作至多 m 次 PUSH，元素工作總成本≤2m；即使某次 MULTIPOP 花 Θ(m)，任意整段序列仍是 O(m)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二進位計數器的總翻轉次數

固定 b≥1 位元 counter 初值全0，INCREMENT 從最低位開始，把連續尾端1翻成0，再把第一個0翻成1；全1則溢位回全0。成本以翻轉次數計。

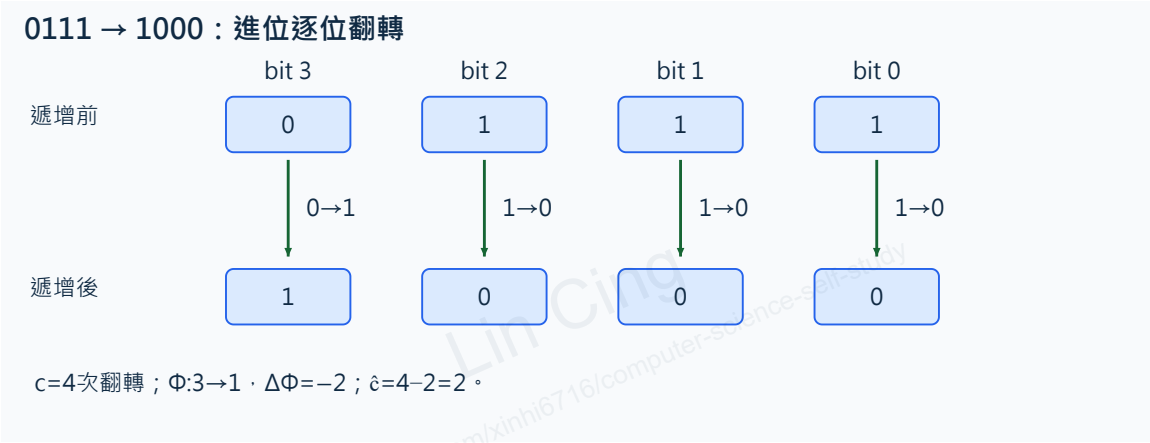
第 j 位每 2^j 次遞增翻轉一次。m 次遞增總翻轉數為

`Σ_(j=0)^(b−1) floor(m/2^j) < 2m`。

不需要假設各位元獨立。位元0的 m 次翻轉也給 Ω(m) 下界，故總成本 Θ(m)。若每次都掃完 b 位才停止，即使高位沒有翻轉也算成本，這個實作就不是本模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二進位計數器的總翻轉次數 （續）



c=4次翻轉；Φ:3→1，ΔΦ=−2；ĉ=4−2=2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

倍增表的幾何和上界

先只考慮插入，起始 (元素數n,容量M)=(0,0)。第一次配置1格；其後滿表時容量倍增。成本定義為1次寫入加搬移舊元素數，配置/釋放的固定開銷計常數。

若插入 m≥2 個元素，最後一次搬移量 L 是小於 m 的最大2冪。所有搬移量為1,2,4,⋯,L，總和 2L−1<2m，故

$T(m)=m+2L-1<3m$。

不能把搬移量一般寫成≤m。m=5時搬移1+2+4=7>5，總成本12；m=8時搬移7，總成本15。只有在某些特定終點，搬移量才剛好小於插入次數。

若新儲存區每格都須初始化，初始化也要計成本；其幾何和仍為 O(m)，但本節精確常數3需相應調整。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.2 The accounting method

會計法先向每操作收一個攤銷費用，收入減實際支出形成信用。要證明所有操作前綴的信用都不負，才能用總收費上界總實際成本。

空 stack 可對 PUSH 收2、POP/MULTIPOP 收0。推入時1支付本次工作，另1附在新元素上；這項元素日後彈出時使用自己的1。因不可能彈出不存在的元素，任何前綴不會透支。若每呼叫另有固定成本，再加固定收費即可。

counter 每次遞增收2。每次0→1先付1翻轉費、另存1在該位元；每次1→0用該位元原有的1支付。溢位全翻回0時全部由舊信用支付。初值全0沒有信用；非零初值則要另給初始1位元足夠信用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

倍增表的收費與真實序列

每插入收3。由前述任意前綴 T(m)<3m，信用 $B_m=3m-T(m)\geq 0$，故收費足夠。下表從空表開始，不把搬移發生位置平移到下一次操作。

次數	操作後(n,M)	搬移量	c_i	累計成本	信用B_i
1	(1,1)	0	1	1	2
2	(2,2)	1	2	3	3
3	(3,4)	2	3	6	3
4	(4,4)	0	1	7	5
5	(5,8)	4	5	12	3
6	(6,8)	0	1	13	5
7	(7,8)	0	1	14	7
8	(8,8)	0	1	15	9

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

倍增表的收費與真實序列 （續）

第3次才是容量2→4；第4次不搬移。這個整體信用帳也可由下一節的勢能不等式逐步證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.3 The potential method

狀態 D_i 的勢能為 $\Phi(D_i)$，定義

$\hat{c}_i=c_i+\Phi(D_i)-\Phi(D_{i-1})$。

對 m 次操作相加，中間勢能完全抵消：

$\sum c_i=\sum \hat{c}_i+\Phi(D_0)-\Phi(D_m)$。

若所有可達終態滿足 $\Phi(D_m)\geq\Phi(D_0)$，就得到 $\sum c_i\leq\sum\hat{c}_i$。常用充分條件是 $\Phi(D_0)=0$、$\Phi(D_i)\geq 0$。若初始勢能為正，則一般是 $\sum c_i\leq\sum\hat{c}_i+\Phi(D_0)$，不能無故丟掉初始項。

勢能整體平移常數不改變 $\hat{c}_i$；真正重要的是末態相對初態的下界，不是函數是否在所有狀態絕對非負。單次 $\hat{c}_i$ 可以負，因先前蓄積的勢能會補貼本次成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stack 與 Counter 的勢能證明

空 stack 取 Φ=size。PUSH 真實成本1、ΔΦ=1，ĉ=2。移除q項的 POP/MULTIPOP 真實成本q、ΔΦ=−q，ĉ=0。Φ始終非負、初值0，立刻得任意 m 次操作的元素成本≤2m。

counter 取 Φ=1位元數。若此次有t個尾端1翻為0，再有一個0翻為1，c=t+1、ΔΦ=1−t，所以 ĉ=2。若全b位為1而溢位，c=b、ΔΦ=−b，所以 ĉ=0。初值全0，故總翻轉≤2m。

若counter起值為任意b位數，Φ(D_0)≤b，得到總翻轉≤2m+b。第一次從全1開始遞增可花b次翻轉，這與該界一致；不能仍宣稱從任意初態開始每個前綴只需2m。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.4 Dynamic tables

現在同時支援插入與刪除末端。容量保持2冪，規則固定如下。

```
insert:
    if M==0: allocate M=1
    elif n==M: allocate 2M; copy n items; M=2M
    write new item; n++
delete-last:
    require n>0
    n--
    if n==0: free storage; M=0
    elif M>=4 and n<=M/4:
        allocate M/2; copy n items; M=M/2
```

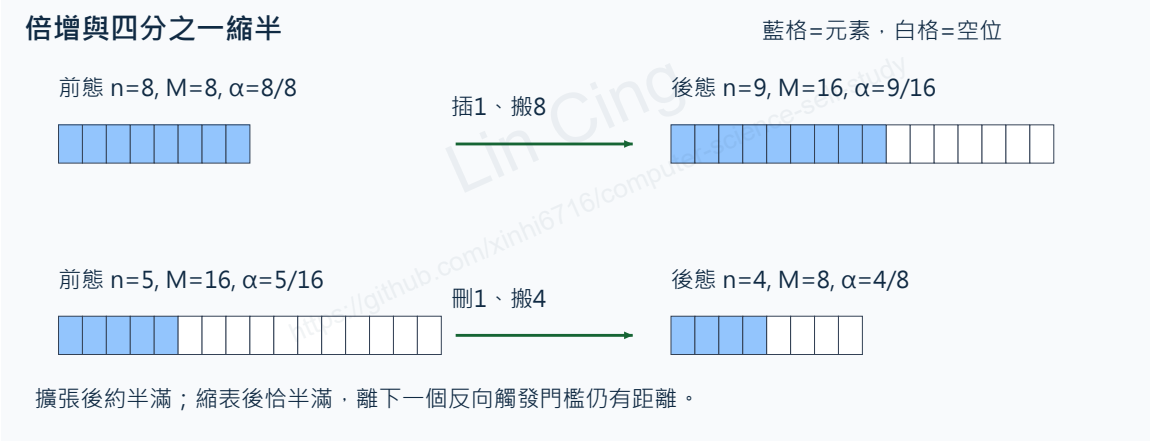
本文用「刪除後n≤M/4就縮半」，與原書「低於1/4」的觸發邊界稍異，因此以下精確數值按本文規則自行證明，不能混用原書邊界的算式。成功刪除的基本成本1，另加縮表搬移n項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何縮表要留緩衝區

若在刪到半滿時就縮半，取滿表(n,M)=(k,k)，k是2冪且足夠大。插入1項，搬k項變(k+1,2k)；刪除1項後n=k，立即縮回M=k，又搬k項。重複此兩步，每輪Θ(k)，m次操作可達Θ(mk)。

改成1/4縮半後，擴表後約半滿，須再刪Θ(M)次才縮；縮表後半滿，須再插Θ(M)次才擴。這提供蓄積信用的空間，下一頁用分段勢能證明所有混合操作序列，而不只用這個直覺。



分段勢能的定義與幾何

空表取Φ(0,0)=0。非空表定義負載率α=n/M，

```
Φ(n,M) = 2n−M      若 n>=M/2
         M/2−n     若 n<M/2
```

兩段在半滿都為0，且各自非負。滿表Φ=M，足以支付下一次搬M項；四分之一滿表Φ=M/4，與縮表搬移量同階。

只用2n−M並不適合允許刪除的版本。例如合法可達的n=3、M=8時為−2，不能套初態0及末態非負的證明。分段勢能在相同狀態是1，且觸發縮表的具體前後態仍需逐案計算。

插入的逐案攤銷界

首次空表插入：(0,0)→(1,1)，c=1、Φ:0→1，ĉ=2。

無擴表且原本至少半滿，ΔΦ=2，因此ĉ=3。原本低於半滿時，新狀態最多恰達半滿；兩態使用低負載線段連續值，ΔΦ=−1，故ĉ=0。

滿表擴張時，操作前(n,M)=(M,M)，操作後(M+1,2M)。真實成本M+1，勢能由M變2，所以

$\hat{c}=(M+1)+(2-M)=3$。

每種插入情況的ĉ≤3。這個推導同時包含「在半滿邊界切換分支」，不假設操作前後一定套同一條式子。

刪除的逐案攤銷界

無縮表且刪後仍至少半滿，ΔΦ=−2，ĉ=−1。若由恰半滿降一項，勢能0→1，ĉ=2；若原本低於半滿而不縮，ΔΦ=1，ĉ=2。

若本次縮表，操作前必為 n_old=M/4+1，刪後n_new=M/4，容量變M/2。操作前Φ=M/4−1，操作後恰半滿Φ=0，真實成本1+M/4，所以

$\hat{c}=1+M/4-(M/4-1)=2$。

刪到空時直接釋放，c=1，Φ由非負值降為0，故ĉ≤1。所有操作的ĉ≤3，初勢能0、末勢能≥0，因此任意m次合法操作的實際成本≤3m。這是確定的最壞序列界，沒有機率假設。

完整混合追蹤與模型邊界

從空表插入至n=5，最後(n,M,Φ)=(5,8,2)。接著刪除：

操作	前態(n,M,Φ)	後態(n,M,Φ)	搬移	c	ĉ
delete	(5,8,2)	(4,8,0)	0	1	−1
delete	(4,8,0)	(3,8,1)	0	1	2
delete	(3,8,1)	(2,4,0)	2	3	2
delete	(2,4,0)	(1,2,0)	1	2	2
delete	(1,2,0)	(0,0,0)	0	1	1

五次插入成本12、五次刪除成本8，總20；攤銷費用總和也為20，因兩端Φ皆0。某些操作ĉ為負不表示實際花負時間。

這裡只刪末端，若刪陣列中間並維持連續順序，另有搬移後綴的Θ(n)成本，不能沿用O(1)攤銷。成長倍率、收縮倍率或門檻更改時，須重新驗證勢能斜率與每個邊界。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 17。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study